

Rewriting the σ -Calculus

Anonymous author

Anonymous affiliation

Abstract

Choosing a representation of variable bindings and substitutions is a central challenge in mechanizing programming language metatheory. De Bruijn indices provide a canonical representation, but developments based on them require numerous lemmas about substitution composition and interaction with binders, leading to significant proof overhead.

The σ -calculus treats parallel substitutions as first-class objects equipped with primitive operations and a terminating and confluent rewrite system that normalizes substitution expressions. While previous work has exploited these rewrite rules to implement decision procedures, the resulting equalities remain propositional.

Recent support for user-defined rewrite rules in mainstream proof assistants enables the promotion of propositional equalities to definitional ones. The rule set must be terminating and confluent and each rule must be supported by a propositional proof, both to keep type checking decidable and to preserve logical consistency. Obtaining these proofs requires modeling substitutions as functions, which in turn leads to violations of the syntactic restrictions on rewrite rules.

In the context of the proof assistant Agda, we embed the rules of the σ -calculus such that they satisfy the criteria for rewrite rules while retaining their provability. This approach yields built-in normalization of substitutions while preserving logical consistency. We evaluate the resulting framework on a multi-sorted, intrinsically scoped formalization of System F, including a proof of subject reduction, and compare proof sizes against an existing Agda development that does not use our infrastructure.

Our development is reusable: from an Autosubst-style higher-order abstract syntax specification, we automatically generate the required σ -calculus infrastructure and rewrite rules. We have used this generator on the example syntaxes from the AUTOSUBST 2 repository. To our knowledge, this is the first safe integration of the σ -calculus rule set with first-class renamings as definitional rewriting in a proof assistant.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Equational logic and rewriting

Keywords and phrases Explicit Substitutions, Rewrite Systems, Agda

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Effectively handling variable bindings and substitutions remains a central challenge when mechanizing programming language metatheory. Compared to pen-and-paper proofs, binders in proof assistants are a notorious source of uninteresting technicalities and significant boilerplate. Since the beginning of proof assistants, many representations have been proposed, including locally nameless binders [1], nominal syntax [35, 36], higher-order abstract syntax, and de Bruijn indices [30]. Choosing the right representation involves genuine tradeoffs. The de Bruijn representation comes with a canonical name for every variable and lets termination of substitution traversals be checked syntactically, but is often considered hard to read for humans, and developments based on it still include substantial proof overhead by requiring numerous lemmas to manage substitution composition and interactions with binders.

While substitution on paper is sometimes presented as an operation that replaces a single variable, mechanizations almost always use parallel substitutions, which map variables to terms and replace multiple variables simultaneously when a term is instantiated with them.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0



Leibniz International Proceedings in Informatics

Schloss Dagstuhl -- Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

1.1 The σ -Calculus in Autosubst

The σ -calculus [1] treats parallel substitutions as first-class objects equipped with primitive operations and a terminating and confluent [32] rewrite system that normalizes substitution expressions. The σ_{SP} [14] variant complements the original to a complete [29] procedure. We refer to the SP variant simply as the σ -calculus throughout this work.

The de Bruijn algebra, where parallel substitutions are modeled as functions, constitutes a semantic model of the σ -calculus. With a semantic model for the substitution operations, the σ -calculus rules become provable, and therefore qualify for equational reasoning in proof assistants [30]. While previous work has exploited these equations to implement automated normalization procedures, the resulting equalities remain propositional. AUTOSUBST [30] provided the first normalization procedure using the σ -calculus rules for unscoped syntax definitions inside the Rocq proof assistant. To normalize terms with parallel substitutions one needs to apply a tactic inside a proof goal.

The successor AUTOSUBST 2 [33] is now regarded as the gold standard for libraries that handle substitution in proof assistants. AUTOSUBST 2 supports intrinsically scoped syntax where the syntax data type is indexed over the number of free variables and variable usage is restricted to the variables in scope by construction. Furthermore, AUTOSUBST 2 generates proofs, for the σ -calculus rules that follow by induction over the syntax, from higher-order abstract syntax descriptions. It also enables reasoning about syntaxes with multiple, mutually dependent, sorts by packaging parallel substitutions over different sorts into vectors.

In particular, AUTOSUBST 2, in contrast to the first version, includes first-class support for renamings, a special case of substitutions that map variables to variables. Independently of first-class support in the equational theory, renamings are required in proof assistants to model substitutions in a structurally recursive manner that the termination checker accepts [3]. The creators of AUTOSUBST 2 extended the equational theory of the σ -calculus with additional rules that handle renamings. The resulting system is not complete anymore, but the authors conjecture that the extended system preserves confluence [32].

1.2 Promoting Propositional to Definitional Equalities with Rewriting

Recent support for user-defined rewrite rules in mainstream proof assistants, such as Agda [11, 13] and Rocq [22], enables the promotion of propositional equalities to definitional ones. To retain logical consistency, the set of equational rules must be confluent and supported by proofs [13]. Agda and other proof assistants perform higher-order pattern unification [2] to detect if a registered rewrite rule can be applied to a stuck term (one that does not reduce further, given the built-in reduction rules). To do so, Agda requires that the left-hand side is a stuck term on a defined symbol.

To model parallel substitution in proof assistants, we have essentially two choices: data types or functions. When we model substitution as functions, some primitive operations of the σ -calculus generate stuck lambdas, thereby violating the syntactic restriction that the left-hand side must be a stuck symbol and not a lambda. The alternative is to model substitutions as a data type, e.g. as a vector of terms with cons and identity as constructors, as is done in AUTOSUBST 2 [33]. While such a representation suffices to mechanize the σ -calculus *propositionally*, it does not give us the full computational behavior required to validate the η -laws of the σ -calculus rule set as *definitional* equalities (because η requires that the cons constructor have computational behavior). Concrete consequences of this limitation are discussed in Section 3.4.

If we wish to rewrite all σ -calculus rules, we require more control over the reduction

behavior of symbols. Fortunately, Agda supports opaque definitions [18], that allow us to selectively block and later unfold the implementation of function definitions. With opaque definitions we can have both: we treat function definitions as stuck symbols, such that we can apply rewrite rules, but also unfold their definitions to prove the rules using their computational behavior.

1.3 Content of this Paper

Contributions

We show how to rewrite the σ -calculus rules, with support for first-class renamings, in a multi-sorted and scoped syntax setting, in Agda. We do so while preserving the logical consistency of the system: every registered rule is proved as an Agda equality before being declared as a rewrite rule, and Agda’s local-confluence checker accepts the resulting rule set.

As an empirical contribution, we provide additional evidence that the extension of the σ -calculus rules with first-class renamings, conjectured to preserve confluence by Stark [32], indeed passes Agda’s local-confluence check on our concrete instantiation. We also demonstrate that the approach translates to a useful class of scoped syntaxes by implementing a code generation script for higher-order abstract syntax descriptions, which we have run on the example syntaxes from the AUTOSUBST 2 repository (Section 4).

We evaluate our approach on a multi-sorted, intrinsically scoped formalization of System F, including a proof of subject reduction, and report a quantitative comparison against an existing Agda development of System F that does not use our infrastructure (Section 3.6).

Structure

Section 2 introduces the required background in Agda and for the σ -calculus. Section 3 demonstrates how to register the σ -calculus rules exemplified by an instantiation for System F. Section 3.6 reports a quantitative comparison with an existing Agda development of subject reduction for System F. We discuss ongoing and future work in Section 4, related work in Section 5 and conclude the paper in Section 6.

2 Preliminaries

We begin with a review of material from the literature. We summarize

- Agda and language features relevant to our work: rewrite rules and opaque blocks;
- multi-sorted and scoped syntax in Agda and
- the σ -calculus modeled via the de Bruijn algebra.

All Agda code presented in this paper can be found in the supplement.

2.1 Agda and Relevant Language Features

Agda is a dependently typed functional language and proof assistant based on Martin-Löf type theory [24], where proofs are verified by type checking. Proofs are significantly easier when terms are *definitionally equal* (i.e., reduce to the same term). When terms are not definitionally equal, users must resort to manual equational reasoning using *propositional equality* (Agda’s identity type, written `==`, with constructor `refl`), which is a cumbersome process because Agda often lacks automated proof-discharge tactics. Maximizing definitional equalities thus directly reduces manual effort.

XX:4 Rewriting the σ -Calculus

Beyond standard features such as indexed inductive types and dependent pattern matching [12], this work is based on two more advanced language features.

Rewrite Rules

User-defined rewrite rules [13, 11] extend Agda’s normalization-by-evaluation approach for definitional equalities by treating propositional equalities as reduction rules. A term is *stuck* if no built-in reduction rule fires on it, e.g. because its head is an Agda **postulate** (an axiomatic constant whose body is not given) or a defined function whose arguments do not yet match any clause. Stuck terms are exactly those candidates against which Agda will try to apply a registered rewrite rule.

A function is eligible for rewriting if it is of the following form:

$$\text{eq} : (x_1 : A_1) \dots \rightarrow f p_1 \dots p_n \equiv v$$

In addition it needs to satisfy the following conditions:

1. The head **f** is a postulate, a defined function, or a data-type or record constructor; if a constructor, its parameters must be distinct bound variables.
2. Every variable x_i appears in a *pattern position* among $p_1, \dots, p_n$, i.e., as an argument of a constructor or a defined symbol applied in a way that allows higher-order pattern unification to recover its binding.¹
3. The left-hand side $f p_1 \dots p_n$ is stuck.

For example, the right identity of addition satisfies these criteria:

$$\begin{aligned} \text{+-id}_r &: \forall n \rightarrow n + 0 \equiv n \\ \text{+-id}_r \text{ zero} &= \text{refl} \\ \text{+-id}_r (\text{suc } n) &= \text{cong suc } (\text{+-id}_r n) \end{aligned}$$

Once the rule is registered via `{-# REWRITE +-idr #-}`, Agda treats $n + 0$ as definitionally equal to n :

$$\begin{aligned} _ &: \forall \{n\} \rightarrow n + 0 \equiv n \\ _ &= \text{refl} \end{aligned}$$

In addition to the syntactic restrictions, users must ensure that the rewrite rule set as a whole is terminating and confluent. Termination is not machine-verified by Agda, but a non-terminating rewrite rule does not by itself introduce inconsistency: it merely renders type checking non-terminating [13]. Confluence, by contrast, is critical for soundness: a non-confluent set of definitional rewrite rules can identify distinct closed normal forms (e.g. **true** and **false**) and thereby make the type theory inconsistent. Agda provides confluence checks via `--confluence-check` and `--local-confluence-check`. Given that the registered rules are supported by propositional proofs and we additionally assume the rewrite system is terminating, local confluence is sufficient to preserve consistency [13].

¹ The Agda documentation defines the pattern-position restriction precisely: <https://agda.readthedocs.io/en/latest/language/rewriting.html>.

Opaque Blocks

Opaque definitions [18] let the user treat code as non-unfolding symbols. This feature is typically used to improve type checking times, because normally we want all definitions to unfold, to produce as many definitional equalities as possible. Their use case in this work will be to declare relevant definitions opaque, such that the stuck symbols qualify for rewriting.

Definitions within an `opaque` block are visible but treated abstractly during type checking:

```
opaque
  forty-two : Nat
  forty-two = 42
```

Outside this block, `forty-two` does not reduce to `42`:

```
_ : forty-two ≡ 42
_ = {! !} -- not provable.
```

However, the `unfolding` keyword can be used to expose the computational behavior of a symbol when needed:

```
opaque
  unfolding forty-two

_ : forty-two ≡ 42
_ = refl
```

2.2 Multi-Sorted and Scoped Syntax

Many programming languages feature multiple syntactic categories. For instance System F [17, 26] has three syntactic sorts: expressions, types, and kinds.² We use intrinsically scoped syntax to prevent the construction of ill-scoped terms from our syntax. Figure 1 (left) shows a standard implementation: types are indexed by the amount of type variables, and expressions by both the amount of type and expression variables. Variables use de Bruijn indices of type `Fin n`, that is, natural numbers strictly less than the number of free variables.

We see scoped syntax as the optimal balance between fully extrinsic syntax and intrinsically typed syntax, in which case the expression syntax is indexed directly over contexts and types. It is powerful enough to detect scoping related errors, but one does not need to commit to full blown intrinsic typing, which can be difficult to work with for more complex systems.

Alas, the standard representation requires us to define many substitution operations across the different sorts (e.g., type-in-type, type-in-expr, expr-in-expr). This representation is chosen by AUTOSUBST 2, and explains why they need vector substitutions, that bundle substitutions over the different sorts alongside additional normalization rules.

To avoid the combinatorial explosion of substitutions over different syntactic sorts, we unify the syntax in two orthogonal dimensions, presented also in Figure 1 (right). Variables and terms form a single datatype $S \vdash [m] s$ indexed by a `Mode` m (taking the values `V` for variables and `T` for terms), a target `Sort` s and a list of sorts `Scope` S . We use the abbreviations $S \ni s := S \vdash [V] s$ for de Bruijn indices and $S \vdash s := S \vdash [T] s$ for terms. Variables are

² Technically kinding for System F is trivial and could be omitted.

XX:6 Rewriting the σ -Calculus

```

data Kind : Set where
  * : Kind

data Type (n : Nat) : Set where
  ' _ : Fin n → Type n
  ∀[α:_] _ : Kind → Type (suc n) → Type n
  _⇒_ : Type n → Type n → Type n

data Expr (n m : Nat) : Set where
  ' _ : Fin m → Expr n m
  λx _ : Expr n (suc m) → Expr n m
  Λα _ : Kind → Expr (suc n) m → Expr n m
  '· _ : Expr n m → Expr n m → Expr n m
  '• _ : Expr n m → Type n → Expr n m

data Sort : Set where
  expr type kind : Sort

Scope = List Sort

data Mode : Set where V T : Mode

data _⊢[_]_ : Scope → Mode → Sort → Set
  _⊢_ = _⊢[ T ]_
  _⊃_ = _⊢[ V ]_

data _⊢[_]_ where
  zero : (s :: S) ⊃ s
  suc : S ⊃ s → (s' :: S) ⊃ s
  ' _ : S ⊃ s → S ⊢ s
  λx _ : (expr :: S) ⊢ expr → S ⊢ expr
  Λα _ : (type :: S) ⊢ expr → S ⊢ expr
  ∀[α:_] _ : S ⊢ kind → (type :: S) ⊢ type → S ⊢ type
  '· _ : S ⊢ expr → S ⊢ expr → S ⊢ expr
  '• _ : S ⊢ expr → S ⊢ type → S ⊢ expr
  _⇒_ : S ⊢ type → S ⊢ type → S ⊢ type
  * : S ⊢ kind

```

■ **Figure 1** Classically Scoped vs. Multi-Sorted Scoped Syntax

inhabitants of $S \ni s$, where the length of S encodes the amount of variables in scope and the position of s in S directly corresponds to the de Bruijn index. So while **zero** and **suc** look like constructors of natural numbers, they are really membership-witness constructors (akin to a **Fin**-valued position index, but extended to multi-sorted scopes).

Using the syntax unified over sorts lets us define renamings and substitutions once for all sorts, avoiding vector substitutions. Unifying over modes serves a more focused purpose: it lets the same instantiation operator $_ \dots^S _$ act on both variables and terms, which is essential later for keeping the variable-lookup case of the σ -calculus rule set as a *stuck* symbol that the rewriter can apply to (see Section 3.4). An alternative design that splits variables and terms into two separate datatypes would still work; we would then need a separate stuck symbol for variable lookup in substitutions.

2.3 The σ -Calculus modeled via the de Bruijn Algebra

We review the variant of the σ -calculus modeled via the de Bruijn algebra which also appears in the first version of AUTOSUBST [30], without first-class renamings. Variables x represent natural numbers and terms $t := x \mid \lambda. t \mid t_1 t_2$ include the standard λ -calculus constructors as an example for any single sorted, unscoped syntax with binding. A substitution σ maps variables to terms. It helps to view a substitution as a list $[t_0, t_1, \dots]$, where the index of the term in the list corresponds to the de Bruijn variable it replaces. Binders make things complicated and require support for a shift operation to increment de Bruijn indices, when substitutions containing variables are pushed under binders. This perspective motivates the

Traversal Rules

$$\begin{array}{ll} \text{lam} & (\lambda. t) \multimap \sigma = \lambda. (t \multimap (\uparrow \sigma)) \\ \text{app} & (t_1 t_2) \multimap \sigma = (t_1 \multimap \sigma) (t_2 \multimap \sigma) \end{array}$$

Monad Rules

$$\begin{array}{ll} \text{right-id} & t \multimap \text{id} = t \\ \text{comp} & (t \multimap \sigma_1) \multimap \sigma_2 = t \multimap (\sigma_1 ; \sigma_2) \end{array}$$

Interaction Rules

$$\begin{array}{lll} \text{id}_L & \text{id} ; \sigma & = \sigma \\ \text{id}_R & \sigma ; \text{id} & = \sigma \\ \text{assoc} & (\sigma_1 ; \sigma_2) ; \sigma_3 & = \sigma_1 ; (\sigma_2 ; \sigma_3) \\ \text{dist} & (t \cdot \sigma_1) ; \sigma_2 & = (t \multimap \sigma_2) \cdot (\sigma_1 ; \sigma_2) \\ \text{interaction} & \text{wk} ; (t \cdot \sigma) & = \sigma \\ \eta\text{-id} & (\text{zero}) \cdot \text{wk} & = \text{id} \\ \eta\text{-law} & (\text{zero} \multimap \sigma) \cdot (\text{wk} ; \sigma) & = \sigma \end{array}$$

■ **Figure 2** The σ -Calculus Rules

semantic definitions for the extension operation $t \cdot \sigma$, the identity id and the weakening wk primitives, treating substitutions as functions:

$$\begin{array}{ll} (t \cdot \sigma)(\text{zero}) := t & \text{id}(x) := x \\ (t \cdot \sigma)(\text{suc } x) := \sigma(x) & \text{wk}(x) := \text{suc } x \end{array}$$

We then define the application of a substitution to a term $\multimap$, which we refer to as instantiation, mutually recursively with the substitution composition operator $;$ (forward composition: apply σ_1 first, then σ_2) and the lifting operator which pushes substitutions under binders using shifting:

$$\begin{array}{ll} x \multimap \sigma = \sigma(x) & (\sigma_1 ; \sigma_2)(x) = \sigma_1(x) \multimap \sigma_2 \\ (\lambda. t) \multimap \sigma = \lambda. (t \multimap (\uparrow \sigma)) & \uparrow \sigma := \text{zero} \cdot (\sigma ; \text{wk}) \\ (t_1 t_2) \multimap \sigma = (t_1 \multimap \sigma) (t_2 \multimap \sigma) \end{array}$$

The variable case --- which Stark calls the only “interesting” case of instantiation --- simply applies the substitution viewed as a function. Lifting preserves the bound variable while it weakens the terms of the substitution to maintain correct scoping. Composition retrieves the term from the first substitution and then instantiates the resulting term with the second. Instantiation traverses the term homomorphically to push the substitution to the leaves. A subtle point: the σ -calculus uses two distinct binary symbols for “composition”, namely the substitution-on-term operator $\multimap$ and the substitution-on-substitution operator $;$; we will reuse this convention throughout.

Figure 2 presents the accompanying rewrite system. It traditionally also includes the clauses of the instantiation function. We follow Stark [32] and categorize the rules into three groups.

- **Traversal rules** govern propagation through terms and under binders.
- **Monad rules** define the behavior of substitutions applied to terms.
- **Interaction rules** describe equalities between substitution expressions.

We refer the reader to the original σ_{SP} -calculus paper [29] for a detailed description of the rule set selection, and to Schäfer et al. [29] for the argument why these rules suffice to normalize all substitution expressions.

3 Rewriting the σ -Calculus

The definition of instantiation by mutual recursion with composition complicates termination. Agda cannot determine that this definition is structurally recursive (and also lacks mechanisms

$$\begin{array}{l}
\text{--}\rightarrow^R\text{--} : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set} \\
S_1 \rightarrow^R S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \ni s \\
\text{opaque} \\
\text{id}^R : S \rightarrow^R S \\
\text{id}^R _ x = x \\
\text{wk}^R : \forall s \rightarrow S \rightarrow^R (s :: S) \\
\text{wk}^R _ _ = \text{succ} \\
\text{--}\circ\text{--} : S_1 \rightarrow^R S_2 \rightarrow S_2 \rightarrow^R S_3 \rightarrow \\
\quad S_1 \rightarrow^R S_3 \\
(\rho_1 \circ \rho_2) _ x = \rho_2 _ (\rho_1 _ x) \\
\text{--}\cdot^R\text{--} : S_2 \ni s \rightarrow S_1 \rightarrow^R S_2 \rightarrow \\
\quad (s :: S_1) \rightarrow^R S_2 \\
(x \cdot^R \rho) _ \text{zero} = x \\
(_ \cdot^R \rho) _ (\text{succ } x) = \rho _ x \\
\text{--}\uparrow^R\text{--} : (S_1 \rightarrow^R S_2) \rightarrow \forall s \rightarrow \\
\quad ((s :: S_1) \rightarrow^R (s :: S_2)) \\
(\rho \uparrow^R _) = \text{zero} \cdot^R (\rho \circ (\text{wk}^R _)) \\
\text{opaque} \\
\text{--}\dots^R\text{--} : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^R S_2 \rightarrow \\
\quad S_2 \vdash [m] s \\
\text{--}\dots^R\text{--} \{m = \mathbf{V}\} x \rho = \rho _ x \\
('x) \dots^R \rho = ' \rho _ x \\
(\lambda x e) \dots^R \rho = \lambda x (e \dots^R (\rho \uparrow^R _)) \\
(\Lambda \alpha e) \dots^R \rho = \Lambda \alpha (e \dots^R (\rho \uparrow^R _)) \\
(\forall [\alpha: k] t) \dots^R \rho = \forall [\alpha: k] \dots^R \rho \\
\quad (t \dots^R (\rho \uparrow^R _)) \\
(e_1 \cdot e_2) \dots^R \rho = (e_1 \dots^R \rho) \cdot (e_2 \dots^R \rho) \\
(e \bullet t) \dots^R \rho = (e \dots^R \rho) \bullet (t \dots^R \rho) \\
(t_1 \Rightarrow t_2) \dots^R \rho = (t_1 \dots^R \rho) \Rightarrow (t_2 \dots^R \rho) \\
* \dots^R \rho = * \\
\text{--}\rightarrow^S\text{--} : \text{Scope} \rightarrow \text{Scope} \rightarrow \text{Set} \\
S_1 \rightarrow^S S_2 = \forall s \rightarrow S_1 \ni s \rightarrow S_2 \vdash s \\
\text{opaque} \\
\langle _ \rangle : S_1 \rightarrow^R S_2 \rightarrow S_1 \rightarrow^S S_2 \\
\langle \rho \rangle _ x = ' \rho _ x \\
\text{id}^S : S \rightarrow^S S \\
\text{id}^S = \langle \text{id}^R \rangle \\
\{-\# \text{ INLINE id}^S \#-\} \\
\text{wk}^S : \forall s \rightarrow S \rightarrow^S (s :: S) \\
\text{wk}^S _ = \langle \text{wk}^R _ \rangle \\
\{-\# \text{ INLINE wk}^S \#-\} \\
\text{opaque} \\
\text{unfolding } \text{--}\dots^R\text{--} \\
\text{--}\dots^S\text{--} : S_2 \vdash s \rightarrow S_1 \rightarrow^S S_2 \rightarrow (s :: S_1) \rightarrow^S S_2 \\
(t \cdot^S \sigma) _ \text{zero} = t \\
(t \cdot^S \sigma) _ (\text{succ } x) = \sigma _ x \\
\text{--}\uparrow^S\text{--} : S_1 \rightarrow^S S_2 \rightarrow \forall s \rightarrow (s :: S_1) \rightarrow^S (s :: S_2) \\
\sigma \uparrow^S s = (' \text{zero}) \cdot^S \lambda _ x \rightarrow (\sigma _ x) \dots^R \text{wk}^R _ \\
\text{--}\dots^S\text{--} : S_1 \vdash [m] s \rightarrow S_1 \rightarrow^S S_2 \rightarrow S_2 \vdash s \\
\text{--}\dots^S\text{--} \{m = \mathbf{V}\} x \sigma = \sigma _ x \\
('x) \dots^S \sigma = \sigma _ x \\
(\lambda x e) \dots^S \sigma = \lambda x (e \dots^S (\sigma \uparrow^S _)) \\
(\Lambda \alpha e) \dots^S \sigma = \Lambda \alpha (e \dots^S (\sigma \uparrow^S _)) \\
(\forall [\alpha: k] t) \dots^S \sigma = \forall [\alpha: k] \dots^S \sigma (t \dots^S (\sigma \uparrow^S _)) \\
(e_1 \cdot e_2) \dots^S \sigma = (e_1 \dots^S \sigma) \cdot (e_2 \dots^S \sigma) \\
(e \bullet t) \dots^S \sigma = (e \dots^S \sigma) \bullet (t \dots^S \sigma) \\
(t_1 \Rightarrow t_2) \dots^S \sigma = (t_1 \dots^S \sigma) \Rightarrow (t_2 \dots^S \sigma) \\
* \dots^S \sigma = * \\
\text{--}\S\text{--} : S_1 \rightarrow^S S_2 \rightarrow S_2 \rightarrow^S S_3 \rightarrow S_1 \rightarrow^S S_3 \\
(\sigma_1 \S \sigma_2) _ x = (\sigma_1 _ x) \dots^S \sigma_2
\end{array}$$

■ **Figure 3** The Primitives for Renaming and Substitution

to justify termination by well-founded recursion), so we follow Adams [3] to resolve the issue by first defining instantiation with renamings. We then extend the equational theory to handle first-class renamings, and also to connect renamings with substitutions, with the rules from AUTOSUBST 2. We use the multi-sorted and scoped version of System F from Figure 1 as an example for any scoped syntax. The approach described in this section directly translates to any multi-sorted and scoped syntax as well as simple intrinsically typed syntax where the type index of the syntax is of type `Set`.

3.1 Renaming and Substitution

The definitions for renaming and substitution are presented in Figure 3. The definitions are mostly standard, but we will block the reduction for most of the renaming and substitution primitives, and describe their behavior later through rewrite rules.

We begin with the type alias for renamings $\text{--}\rightarrow^R\text{--}$: renamings are functions that map

variables of sort s from scope S_1 to variables of the same sort in scope S_2 . We then define the renaming primitives identity id^R , weakening wk^R and *forward* composition $_ \circ _$, where $(\rho_1 \circ \rho_2) x = \rho_2 (\rho_1 x)$. We use forward composition (rather than the more common mathematical $(f \circ g) x = f(g x)$) to match the σ -calculus tradition where substitution composition $_ \circ _$ also reads left-to-right; the operator name $\circ$ is unfortunate in this respect, but we keep it for consistency with prior AUTOSUBST developments. The identity renaming is just the identity function, ignoring the explicit sort argument, which we cannot leave implicit because Agda often fails to infer it, complicating proofs unnecessarily (we present the source code with this argument explicit even though, conceptually, it should be inferable). Weakening applies the succ constructor to all variables of a renaming. Extension $_ \cdot^R _$ inserts the respective variable for de Bruijn index zero into the renaming. Lifting $_ \uparrow^R _$ is defined in terms of extension with variable zero and composition with the weakening renaming. We do not mark lifting opaque, because it is merely an alias for a term constructed from other primitives. The traversal operation $_ \dots^R _$ propagates the renaming through the nodes to the leaves and applies lifting when it passes under binders. We give a single definition that handles both $\mathbf{V}$ - and $\mathbf{T}$ -mode arguments uniformly: the $\mathbf{V}$ -mode case --- which is the only “interesting” case for variables --- is the first clause $_ \dots^R _ \{m = \mathbf{V}\} x \rho = \rho x$. We define instantiation across both modes for reasons explained later.

Substitutions are functions from variables to terms instead of variables. The operator $\langle _ \rangle$ embeds any renaming into a substitution by wrapping all variables with the variable constructor. With the embedding operator we define aliases for the identity substitution id^S and weakening substitution wk^S directly from renamings. To prevent Agda from rejecting rewrite rules due to reductions on the left-hand side of equations, we **INLINE** these symbols.

Extension $_ \cdot^S _$ is defined by case analysis on the incoming variable and maps the term we extend with to variable zero . We use instantiation with the weakening renaming to define the lifting of substitutions $_ \uparrow^S _$, thereby satisfying the requirements of structural recursion. In contrast to the lifting operator for renamings, we keep its computational behavior hidden, and later rewrite it in terms of substitution composition. Instantiation with a substitution $_ \dots^S _$ works analogous to the case of renamings and we also design it to work for variables and terms simultaneously. Finally we define the forward composition of substitutions $_ \circ _$ via instantiation.

The definition of instantiation across both modes lets us perform lookup of a variable in a substitution (or renaming) using the stuck symbol for instantiation ($x \dots^S \sigma$) rather than via function application ($\sigma _ x$). Because we need to state the σ -calculus rules only in terms of stuck symbols --- never via raw lambdas or function application --- we require an additional symbol for variable lookup; the stuck $_ \dots^S _$ on a variable is exactly this symbol. We go into more detail in Section 3.4.

3.2 The σ -Calculus Rules

Figure 4 presents the part of σ -calculus rules with first-class renamings as Agda equalities admissible for rewriting. The figure should be read first column then second column, all rule *statements* appearing above the **REWRITE** block; the names appearing inside the **REWRITE** pragma are those declared above. The second paragraph of the **REWRITE** block --- the analogous renaming-only rules --- was omitted from the figure and can be found in the supplement.

Agda accepts the rule set as locally confluent, which together with our termination assumption suffices for safety (Section 2.1). All rules are proved as Agda equalities before rewriting, ensuring that the new definitional equalities are sound with respect to the underly-

XX:10 Rewriting the σ -Calculus

```

-- definitional rules
def-S-zero : zero ...S (t .S σ) ≡ t
def-S-suc : suc x ...S (t .S σ) ≡ x ...S σ
def-S-; : (x ...S (σ1 ; σ2)) ≡ ((x ...S σ1) ...S σ2)
def-S-↑ : σ ↑S s ≡ (' zero) .S (σ ; wkS _)

-- interaction rules
assoc : (σ1 ; σ2) ; σ3 ≡ σ1 ; (σ2 ; σ3)
dist : (t .S σ1) ; σ2 ≡ ((t ...S σ2) .S (σ1 ; σ2))
interact : wkS s ; (t .S σ) ≡ σ
comp-idr : σ ; idS ≡ σ
comp-idl : idS ; σ ≡ σ
η-id : (' zero {s} {S}) .S (wkS _) ≡ idS
η-law : (zero ...S σ) .S (wkS _ ; σ) ≡ σ

-- complete rewrite system
{-# REWRITE
def-S-zero def-S-suc def-S-↑ def-S-;
assoc dist interact
comp-idr comp-idl η-id η-law
inst-x inst-λ inst-Λ inst-∀ inst-· inst-•
inst-⇒ inst-*
right-id
compositionalityRR compositionalityRS
compositionalitySR compositionalitySS
coincidence coincidence-fold

def-id def-wk def-R-zero def-R-suc def-○
assocR distR interactR
comp-idrR comp-idlR η-idR η-lawR
instR-x instR-λ instR-Λ instR-∀ instR-· instR-•
instR-⇒ instR-*
coincidence-var
#-}
```

```

-- coincidence rules
coincidence : ∀ (t : S ⊢ s) →
  t ...S ⟨ ρ ⟩ ≡ (t ...R ρ)
coincidence-fold :
  ⟨ ρ ↑R s ⟩ ; ((t ...R ρ) .S idS) ≡ (t ...R ρ) .S ⟨ ρ ⟩

-- traversal rules
inst-x : (' x) ...S σ ≡ x ...S σ
inst-λ : (λx e) ...S σ ≡
  λx (e ...S (σ ↑S _))
inst-Λ : (Λα e) ...S σ ≡
  Λα (e ...S (σ ↑S _))
inst-∀ : (∀[α: k] t) ...S σ ≡
  ∀[α: k ...S σ] (t ...S (σ ↑S _))
inst-· : (e1 · e2) ...S σ ≡
  (e1 ...S σ) · (e2 ...S σ)
inst-• : (e • t) ...S σ ≡
  (e ...S σ) • (t ...S σ)
inst-⇒ : (t1 ⇒ t2) ...S σ ≡
  (t1 ...S σ) ⇒ (t2 ...S σ)
inst-* : * ...S σ ≡ *

-- monad rules
right-id : ∀ (t : S ⊢ s) → t ...R idR ≡ t
compositionalityRR : ∀ (t : S ⊢ s) →
  (t ...R ρ1) ...R ρ2 ≡ t ...R (ρ1 ○ ρ2)
compositionalityRS : ∀ (t : S ⊢ s) →
  (t ...R ρ1) ...S σ2 ≡ t ...S (⟨ ρ1 ⟩ ; σ2)
compositionalitySR : ∀ (t : S ⊢ s) →
  (t ...S σ1) ...R ρ2 ≡ t ...S (σ1 ; ⟨ ρ2 ⟩)
compositionalitySS : ∀ (t : S ⊢ s) →
  (t ...S σ1) ...S σ2 ≡ t ...S (σ1 ; σ2)

```

■ **Figure 4** The Rewrite System For the σ -Calculus with First-Class Renamings

ing intensional Agda theory. We now discuss the changes to the standard σ -calculus variant from Section 2.3.

The **Definitional rules** include definitions we blocked earlier in modified form. We include the clauses from the definition of extension but use the instantiation operator to handle lookup of variables in substitutions instead of function application. Similarly, lookup in composed substitutions is defined in terms of the instantiation operator. We rewrite the lifting operator in terms of composition with the weakening substitution, which was not possible in the original definition because of the aforementioned termination issues. This change allows us to draw on the complete theory of the σ -calculus when reasoning solely about substitutions. To save space, we have omitted the definitional rules for renamings, but they follow analogously.

The **Interaction rules** are exactly those presented in Section 2.3. In the figure we have omitted a copy of the interaction rules for renamings, which we also include in the rewrite

system.

The **Coincidence rules** emerge from the interaction between renamings and substitutions [33]. They allow us to recover some renamings from their embedding as substitutions. The extraction of renamings from deeply nested substitutions in general requires a dedicated solving strategy [33]. In addition to the obvious **coincidence** rule, we require a specific coincidence variant **coincidence-fold** for the proof of subject reduction for System F later. Other proofs may require additional coincidence lemmas; in general, proving subject reduction for an object language with binders of maximum arity n requires lemmas in the style of **coincidence-fold** with up to n extensions. We have empirically verified that adding such variants for arities up to 5 to our rewrite system still passes Agda’s local-confluence check; the rules are all left-linear instances of substitution-equality extensions and we are not aware of a configuration where adding a coincidence variant breaks confluence, but we know of no general theorem to that effect.

The **Traversal rules** mirror the clauses from the definitions of instantiation in the previous section directly. We include all clauses except the one for variables. This exclusion is important because we wanted the term $x \dots^S \sigma$ to remain stuck, so we can apply rewrite rules where instantiation of variables is the head symbol. We have omitted the analogous traversal rules for renamings.

We extend the **Monad rules** to handle all four combinations of instantiation with renamings and substitutions. We state the right identity law only for renamings because the instantiation with the identity substitution reduces to instantiation with the identity renaming via **coincidence**.

The proofs of the rules are standard, and we refer the interested reader to the supplement. Within the proofs we unfold the opaque definitions and make use of function extensionality. Function extensionality is needed because the σ -calculus interaction rules equate two substitution *functions*, e.g. $\text{assoc} : (\sigma_1 \circ \sigma_2) \circ \sigma_3 \equiv \sigma_1 \circ (\sigma_2 \circ \sigma_3)$, and these are equal only pointwise. We add function extensionality as a postulate; it is independent of intensional Martin-Löf type theory but is well-known to be *consistent* with Agda [19], so the addition does not threaten consistency. (We are not claiming the addition is conservative.)

3.3 An Example Proof with the Rewritten σ -Calculus

The rules of the σ -calculus apply automatically throughout the subsequent code. We briefly demonstrate this with a proof of subject reduction where we never need to explicitly manage substitutions. The proof is carried out for a standard typing relation $\vdash _ : _$ and call-by-value small-step operational semantics $\hookrightarrow$. We refer to the supplement for the definitions of both.

To establish subject reduction, we first prove that renaming and substitution preserve typing. The left side of Figure 5 displays the proofs for the substitution-preserves-typing lemma $\vdash \dots^S _$. We omit the analogous proof for renamings.

Reading Figure 5. We use a single typing turnstile $\vdash$ (rendered black) for the object-language judgement $\vdash _ : _$. The shorter green-rendered constructors $\vdash'$, $\vdash \lambda$, $\vdash A$, $\vdash \cdot$, $\vdash \bullet$, $\vdash^*$ are the constructors of that judgement (one per term constructor of System F); they are the data of typing derivations. The case for type application, written $\vdash \bullet \vdash e \vdash t \vdash t'$, binds three sub-derivations: $\vdash e$ for the polymorphic expression, $\vdash t$ for the type argument, and $\vdash t'$ for the well-kindedness of the body of the universal type. The variable σ that appears in two places of each clause is a single binding: an *implicit* parameter of the lemma’s signature, and the *explicit* argument inside the proof body --- it is not bound twice.

The signature of the proof relies on the definition of well typed substitutions:

XX:12 Rewriting the σ -Calculus

$$\begin{array}{l}
\text{---} \vdash \dots^S \text{---} : \\
\Gamma_1 \vdash t : t' \rightarrow \\
(\sigma : S_1 \rightarrow^S S_2) \rightarrow \\
\sigma : \Gamma_1 \rightarrow^S \Gamma_2 \rightarrow \\
\Gamma_2 \vdash (t \dots^S \sigma) : (t' \dots^S \sigma) \\
(\vdash' \vdash x) \quad \vdash \dots^S [\sigma] \vdash \sigma = \\
\vdash \sigma \text{ --- } \vdash x \\
(\vdash \lambda \vdash e) \quad \vdash \dots^S [\sigma] \vdash \sigma = \\
\vdash \lambda (\vdash e \vdash \dots^S [\sigma \uparrow^S \text{---}]) (\vdash \uparrow^S [\sigma] \vdash \sigma) \text{---} \\
(\vdash \Delta \vdash e) \quad \vdash \dots^S [\sigma] \vdash \sigma = \\
\vdash \Delta (\vdash e \vdash \dots^S [\sigma \uparrow^S \text{---}]) (\vdash \uparrow^S [\sigma] \vdash \sigma) \text{---} \\
(\vdash \cdot \vdash e_1 \vdash e_2) \quad \vdash \dots^S [\sigma] \vdash \sigma = \\
\vdash \cdot (\vdash e_1 \vdash \dots^S [\sigma] \vdash \sigma) (\vdash e_2 \vdash \dots^S [\sigma] \vdash \sigma) \\
(\vdash \bullet \vdash e \vdash t \vdash t') \quad \vdash \dots^S [\sigma] \vdash \sigma = \\
\vdash \bullet (\vdash e \vdash \dots^S [\sigma] \vdash \sigma) (\vdash t \vdash \dots^S [\sigma] \vdash \sigma) \\
(\vdash t' \vdash \dots^S [\sigma \uparrow^S \text{---}]) (\vdash \uparrow^S [\sigma] \vdash \sigma) \text{---} \\
\vdash^* \quad \vdash \dots^S [\sigma] \vdash \sigma = \vdash^*
\end{array}
\quad
\begin{array}{l}
\text{sr} : \\
\Gamma \vdash e : t \rightarrow \\
e \hookrightarrow e' \rightarrow \\
\Gamma \vdash e' : t \\
\text{sr} (\vdash \cdot \{e_2 = e_2\} (\vdash \lambda \vdash e_1) \vdash e_2) (\beta\text{-}\lambda \ v_2) = \\
\vdash e_1 \vdash \dots^S [e_2 \cdot^S \text{id}^S] (\vdash \square \vdash e_2) \\
\text{sr} (\vdash \bullet \{t = t\} (\vdash \Delta \vdash e) \vdash t \vdash t') \beta\text{-}\Delta = \\
\vdash e \vdash \dots^S [t \cdot^S \text{id}^S] (\vdash \square \vdash t) \\
\text{sr} (\vdash \cdot \vdash e_1 \vdash e_2) (\xi\text{-}\cdot \ 1 \ e_1 \hookrightarrow e) = \\
\vdash \cdot (\text{sr} \vdash e_1 \ e_1 \hookrightarrow e) \vdash e_2 \\
\text{sr} (\vdash \cdot \vdash e_1 \vdash e_2) (\xi\text{-}\cdot \ 2 \ e_2 \hookrightarrow e \ x) = \\
\vdash \cdot \vdash e_1 (\text{sr} \vdash e_2 \ e_2 \hookrightarrow e) \\
\text{sr} (\vdash \bullet \vdash e \vdash t \vdash t') (\xi\text{-}\bullet \ e \hookrightarrow e') = \\
\vdash \bullet (\text{sr} \vdash e \ e \hookrightarrow e') \vdash t \vdash t'
\end{array}$$

■ **Figure 5** Subject Reduction for System F

$$\text{---} : \text{---} \rightarrow^S \text{---} : S_1 \rightarrow^S S_2 \rightarrow (\Gamma_1 : \text{Ctx } S_1) \rightarrow (\Gamma_2 : \text{Ctx } S_2) \rightarrow \text{Set}$$

Contexts $\text{Ctx } S$ are maps from scoped variables to scoped types. Well-typed substitutions are functions from variables x typed in Γ_1 to terms $x \dots^S \sigma$ typed in Γ_2 .

The preservation result for renamings is used to prove that the lifting of a substitution preserves typing $\vdash \uparrow^S$, and subsequently used inside the substitution-preserves-typing proof, again respecting the termination constraints. The variable case requires a lookup of the well-typed variable inside the well-typed substitution. We apply the preservation lemma for lifting substitutions when we pass under a binder. All cases follow without any manipulation of substitutions. In particular we do not need to manually move the substitution under a weakening inside the cases $\lambda x \text{---}$ and $\Delta \alpha \text{---}$ or under a singleton substitution in the case of type application $\text{---} \bullet \text{---}$, which are normally required in standard proofs.

Subject reduction sr directly follows from the substitution-preserves-typing lemma, shown on the right of Figure 5. Inside the β -reduction cases $\vdash \square$ denotes the well-typed single substitution, which can be derived directly from the substitution-preserves-typing lemma. The rewrite rules automatically handle the otherwise required application of interaction rules in the β -reduction cases.

3.4 An Examination of the Issue with Unfolding Definitions

Blocking the reduction behavior of renaming and substitution primitives allowed us to successfully rewrite all σ -calculus rules. We now explain why this approach works, and more specifically why other attempts did not work.

The primary issue is that composition unfolds to λ -abstractions. If we unfold the definition for the composition operator, then for instance the left hand side of comp-id_l rule would reduce to $(\lambda \text{---} x \rightarrow \sigma \text{---} x \dots^S (\lambda \text{---} \rightarrow \text{---}))$, violating the constraint that rewrite rules must have stuck symbols on the left-hand side. However, when we block the reduction of composition, we still need to be able to describe how variables are looked up inside two composed substitutions.

We tried to keep using function application to describe variable lookup, that is, rewriting the following: $(\sigma_1 ; \sigma_2) _ x \equiv (x \dots^S \sigma_1) \dots^S \sigma_2$. This equation effectively re-establishes the computational behavior of composition and to our surprise, Agda accepts this rule together with the interaction rules, such as `comp-idl`, but it fails to perform higher-order pattern unification correctly afterwards. More specifically, it does not apply the η -laws anymore, even though manual application using propositional equality reasoning works. We argue Agda should reject such a construction: registering a rule whose left-hand side has a lambda-applied head (here, a substitution function applied to a variable) violates the documented stuck-symbol restriction, and silently accepting such rules causes higher-order pattern unification to silently miss matches against rules whose left-hand sides are the canonical stuck symbol; since this matching is the soundness boundary between propositional and definitional equality in our setting, accepting such rules undermines the user’s mental model of which equalities will actually fire.

Instead we use the instantiation operator to perform variable lookup. Doing so requires us to propagate this change through all rules, using instantiation every time we wanted to use function application for variable lookup.

We note that, while we are still able to use function application or build renamings and substitutions using lambdas, we then obtain fewer definitional equalities. We thus recommend only to use the instantiation operator for variable lookup and only the two already defined composition operators.

Using Vectors instead of Functions

We also considered an alternative representation for substitutions based on a vector-like data type. This data type would at least need to include an extension constructor, to fill the vector with terms. Because composition is then defined via case distinction on the vector, it correctly behaves as a stuck symbol suitable for rewriting. However, with extension as constructor the η -rules are not instantiable with proofs, because this would require that the extension constructor has computational behavior.

Removing the η -rules from the equational theory preserves confluence, but removes important definitional equalities. However, the vector representation may still be useful in settings where substitutions cannot be modeled as functions --- for instance, in QIIT-based formalisations of type-theory-in-type-theory [7] that use *explicit* substitutions as constructors of the syntax. We do not claim that all approaches to type-theory-in-type-theory rule out function-based substitutions; the recent strictification approach of Kaposi and Pujet [21] (discussed in Section 5) takes the opposite route.

3.5 Discussion

The rules presented in Section 3.2 are an extension of the classic σ -calculus rules to a setting with renamings, closely following the work done by the AUTOSUBST 2 authors. After careful construction, marking the relevant symbols opaque, they fulfill the requirements to be registered as rewrite rules. Moreover Agda’s local-confluence checker accepts the rule set.

The original σ -calculus treats the symbols purely syntactically, also describing the behavior of variable lookup using the instantiation operator. The de Bruijn algebra subsequently provided a semantic model for the σ -calculus using functions, enabling proofs of the σ -calculus rules and their use in automated propositional-equality reasoning. In this work we do both: we use the semantic model to *prove* the laws (inside opaque blocks), and treat the symbols syntactically (outside the opaque blocks) so that the rewriter can predictably *apply* them.

```

kind : Type
type : Type
expr : Type

-- the constructors for kind
star : kind

-- the constructors for expr
app  : expr -> expr -> expr
lam  : type -> (expr -> expr) -> expr
tapp : expr -> type -> expr
tlam : kind -> (type -> expr) -> expr

-- the constructors for type
arr : type -> type -> type
all : kind -> (type -> type) -> type

```

■ **Figure 6** Higher-order abstract syntax specification for System F

3.6 Quantitative Evaluation

To support the claim that rewriting the σ -calculus rules actually reduces proof effort on a non-trivial example, we compare our subject reduction proof for System F with the corresponding development in Stucki’s publicly available extrinsically-scoped Agda formalisation of System F_<: [34] that uses the agda-stdlib `Data.Fin.Substitution` library (a McBride-style generic renaming/substitution framework [3, 25]). Table 1 summarises the comparison; line counts include only the relevant lemmas, not data type declarations or imports.

Component	this work	sstucki/system-f-agda
Substitution-preserves-typing ($\underline{\vdash} \dots^S \underline{}$)	18 lines	---
Subject-reduction proof body	16 lines	27 lines
Substitution lifting / context interaction lemmas	5 lines	≈ 19 lemmas, ≈ 426 lines (<code>WtTerm.agda</code>)
Manual transports / coercions inside the proof	0	several (<code>subst</code> , <code>cong</code> -chains in helpers)

■ **Table 1** Proof-size comparison for subject reduction of System F.

The core case-split in both proofs is comparably short --- this is expected because the β -cases of subject reduction are inherently the “interesting” cases. The difference is that `sstucki/system-f-agda` sustains its 27-line preservation proof on top of hundreds of lines of hand-rolled propositional substitution infrastructure (lifting, weakening, commutation lemmas, well-typedness lifting in `WtTerm.agda`); we replace that infrastructure by definitional rewriting against the σ -calculus rule set. The point is not that our top-level proof is shorter (it is, modestly), but that the supporting cast of “boilerplate” substitution lemmas all but disappears: the user writes the σ -calculus rules *once* (and we generate them automatically from a HOAS specification, see Section 4), and they apply silently for every subsequent metatheoretic proof on the same syntax.

We do not claim a proof-time speedup: type checking the `systemF` case study takes a few seconds on commodity hardware (Intel i7 at 2.5 GHz, 16 GB RAM, Agda 2.6.4), with the `rewrite-system` declaration dominating the cost. Scaling the rewrite set to richer object languages (e.g. dependent types) is open work; we comment on it in Section 4.

4 Ongoing and Future Work

4.1 Code Generation

In addition to the theoretical work, AUTOSUBST 2 also features external code generation from higher-order abstract syntax (HOAS) descriptions. More specifically, they generate the scoped syntax data types and all substitution lemmas that require induction over the syntax for the Rocq proof assistant.

To showcase our applicability to custom syntax descriptions, we also provide a code generation script that translates HOAS descriptions to Agda code, which includes the σ -calculus rules ready for rewriting. The corresponding HOAS description for our System F example can be found in Figure 6. We are already able to support some of the features that Autosubst 2 supports:

- Vector substitutions are unnecessary due to multi-sorted syntax.
- Mutual recursive syntax definitions work out of the box with multi-sorted syntax.
- Binders that bind multiple variables are supported by a generalized lifting operation that allows to lift by lists of sorts, instead of only a single sort.
- Object-language constructors that wrap arbitrary Agda values (e.g. a syntactic node carrying a metalevel `List` of subterms, or a metalevel pair of subterms) can already be encoded by introducing an extra sort that “stands in” for the inline Agda type. The resulting object syntax then mentions that sort instead of the original Agda type, so users lose direct access to the standard library lemmas (e.g. list functions like `map`, `++`) and would have to re-prove the corresponding properties on the wrapping sort.
- Distinction between sorts with binders and those without is not yet supported, but could be encoded with an additional binary index to the `Sort` data type, that separates sorts of the two categories.
- Variadic syntax is also not yet supported, but could be encoded in the future using arguments to the sort constructors [27]

Our code generation script can be found in the supplement, but represents merely a proof of concept for a standalone library. The script includes a parser for HOAS descriptions and ad-hoc code generation. It is written in Python with strict type-checking enabled and includes all the boilerplate Agda code, so it can serve as a single-file standalone tool for others to try. We include a dozen Agda files which we generated from HOAS descriptions found in the examples directory of the AUTOSUBST 2 git repository.

4.2 Opaque Definitions for Rewriting

The construction from the previous section reveals a broader concept that could serve as a general recipe for integrating normalization for suitable algebraic structures into proof assistants:

- Define symbols for the structure.
- Establish a corresponding confluent rewrite system.
- Instantiate the structure with an underlying model to prove the laws of the rewrite system inside `opaque` blocks.
- With the reduction of the model hidden, `REWRITE` the algebraic equations.

To illustrate this, we consider the example of classical sets. When reasoning over sets in Agda, users often resort to work with lists up to permutation and a uniqueness predicate,

which is tedious to work with and requires lots of uninteresting code. This overhead is mostly not an issue in tactic-based provers, that feature dedicated solvers for this work.

The characteristic function for sets makes it possible to translate statements about sets to statements about functions. We briefly outline how our approach could be adapted to provide built-in normalization for sets using their characteristic function as the underlying model.

Following the idea above, the type of sets would be defined as: $\mathcal{S} = \mathcal{A} \rightarrow \text{Bool}$ for some $\mathcal{A} : \text{Set } \ell$. Using this model we can define set operations as opaque functions:

```
opaque
   $\emptyset : \mathcal{S}$ 
   $\emptyset \ a = \text{false}$ 

   $\_ \cup \_ : \mathcal{S} \rightarrow \mathcal{S} \rightarrow \mathcal{S}$ 
   $(A \cup B) \ a = A \ a \vee B \ a$ 

  -- ...
```

Although this model is probably even worse to work with than lists up to permutation, we can define all set operations and also instantiate a suitable rewrite system with proofs. This rewrite system might include for example a right identity law of the form: $(A \cup \emptyset) \equiv A$. Due to the opaquely defined operations, this equation qualifies as a rewrite rule.

A good starting point for a suitable rewrite system is given by the boolean ring interpretation of sets [20]. This system however, requires rewriting modulo AC (associativity and commutativity). Other provers, such as Dedukti [8], which is purely based on rewrite rules, already support AC rewriting and we argue that Agda would also greatly benefit from an extension with rewriting modulo AC. Safe built-in normalization for common algebraic structures would improve the user experience of Agda and might allow it to close the gap to tactic-based provers regarding automation of such proofs.

4.3 Implications for Intrinsically Typed Syntax

Complex intrinsically typed syntax, where substitution already appears as an operation on the type index of the syntax, complicates proofs significantly. Because substitutions without rewriting enjoy few definitional equalities, we need to apply general properties of the transfer lemma (`subst` in Agda) every time we manipulate proofs over expressions that involve type substitution. This issue is commonly referred to as transfer hell (or transport hell).

The fact that complex intrinsically typed syntax ends up in transfer hell is a long-standing issue [9]. An elaborate example close to our work in Agda is given by the construction of a binary logical relation for intrinsically typed System F [28], where the authors spend thousands of lines reasoning about uninteresting manipulation of transfers. More specifically, while reasoning over type-in-type and expr-in-expr substitutions is still easy in this scenario, proving lemmas over type-in-expr substitution is complicated tremendously, because type substitution also affects the index of the expression that is substituted in. We expect that rewriting the σ -calculus rules on the type index of the syntax should remove the bulk of these transfer-hell obstructions; we have not yet ported that development.

Rewriting the σ -calculus rules on the syntax for the typing index should generalize to other intrinsically typed formalizations and render most explicit transfers unnecessary. A very recent case study by Chen, Nordvall Forsberg and Tsai [10] confirms that “transport hell” resurfaces even in Cubical Agda when the metatheory of an intrinsic type theory becomes

sophisticated, suggesting that machinery in the spirit of this paper is needed in addition to higher-order type-theoretic tools.

5 Related Work

We already discussed AUTOSUBST and related research [33, 30, 32], as well as user-defined rewrite rules [13, 11, 22] and opaque definitions [18], both of which are present in Agda and Rocq, extensively throughout this work.

Strictification and Type Theory in Type Theory

Kaposi and Pujet [21] attack a related problem with a very different technique. Their goal is to formalise canonicity for a dependent type theory inside intensional Agda, where the standard obstacle is again “transport hell”: substitution laws on the initial Category-with-Families (CwF) hold only propositionally, so syntactic manipulations drag transports everywhere. Their fix is a *strictification*: they construct an alternative presentation of the initial CwF in which substitutions are encoded as metatheoretic functions (exploiting that function composition is definitionally associative in Agda’s metatheory), so the substitution-calculus equations hold definitionally by construction in plain Agda, with no user-defined rewrite rules. Their work and ours are complementary rather than overlapping. (i) Scope: they target one fixed dependent type theory and one fixed semantic presentation (the initial CwF for Π -types and booleans); we provide a generic infrastructure for arbitrary multi-sorted scoped syntaxes via AUTOSUBST-style code generation. (ii) Mechanism: they re-engineer the syntax inside vanilla Agda so that the chosen substitution equalities follow definitionally from function composition; we keep the natural de Bruijn syntax and instead register the full σ -calculus rule set --- including η and the rules for first-class renamings --- as user-defined REWRITE rules inside opaque blocks. (iii) Rule coverage: their approach handles substitution laws derivable from the initial-CwF reformulation; ours handles the entire σ -calculus rule set with renamings (Figure 4), including coincidence rules. Notably, Kaposi and Pujet themselves cite Cockx et al.’s rewrite-rule machinery as an alternative route they chose not to take; the present paper is essentially the development of that alternative route for general scoped syntaxes.

Local and Global Rewrite Rules in Rocq

In their recent work on local rewrite rules [23], Leray and Winterhalter also rewrite σ -calculus rules with support for first-class renamings in Rocq, similar to our approach. They model renamings and substitutions as functions, but their setting only lets them register a strict subset of the σ -calculus laws as definitional equalities. The restriction is intrinsic to local rewrite rules: in LRTT the equations must hold strictly under their abstraction interface, and the set of equations that can be made strict on the user side is limited by what can be locally validated. Our setting differs because we use Agda’s *global* user-defined rewrite rules [11, 13], which lets us register the full rule set of the σ -calculus as definitional once and for all, after we proved each rule propositionally. Rocq also offers global rewrite rules [22], and we expect our approach translates with no fundamental obstacles --- the Rocq facility likewise requires a propositional justification for each rule, so the same prove-then-register methodology applies. Local versus global is mostly a trade-off: global rules see by everyone (including unrelated developments that import the module), local rules are only in scope for the proof that opens them.

TeaLeaves

Tealeaves [15, 16] is a generic Coq/Rocq library for abstract-syntax metatheory built on a categorical abstraction called a decorated traversable monad (DTM), whose equational theory replaces tedious induction on terms with reusable algebraic laws. End users instantiate one of several backends (de Bruijn, locally nameless) to their grammar and inherit substitution operations and lemmas for free. The contrast with us is along two axes. First, the host system: Tealeaves lives in Coq with propositional equality, so its substitution lemmas are $=$ -equalities you must **rewrite** with in tactic scripts, whereas we make σ -calculus equalities *definitional* via Agda REWRITE rules so that they fire silently during typechecking. Second, the abstraction: Tealeaves abstracts syntax algebraically through a monadic structure that provides high-level reasoning principles, while we keep the concrete de-Bruijn / function-as-substitution view of Autosubst and orient the σ -calculus as a confluent term-rewriting system that Agda’s local-confluence checker validates. The two approaches are complementary: a monadic metalanguage like Tealeaves could in principle generate the propositional σ -calculus rules that we then register as REWRITE rules.

Generic Programming for Syntax with Binding

Another line of research concerns generic programming approaches [4, 5] that allow the construction of syntax from general building blocks supporting sums, products and binder introduction. This allows for a generic structure that abstracts over renamings and substitutions. The generic syntax approach translates to a scoped and multi-sorted setting [27]. Importantly, we believe generic programming and rewriting the σ -calculus rules are orthogonal: our rewrite system could be applied to the universe of syntaxes with binding, thereby avoiding external code generation.

The ABSTRACT BINDING TREES library for Agda shares a similar goal with AUTOSUBST and is used for instance in Siek’s work [31]. It permits the construction of an object language from a generic syntax similar to the universe of syntaxes with binding, but it only supports extrinsic scoping and single-sorted syntaxes. The library mainly focuses on generic theorems about substitution and abstractions for arbitrary predicates over the object language. Interestingly, the library also offers incomplete experimental support for rewriting using σ -calculus laws but misses rules of the σ -calculus and thus does not pass the confluence check.

Explicit Weakening and Uniform Renaming/Substitution

Wadler experiments with explicit substitutions and implements weakening as a constructor in the syntax [37], thereby successfully avoiding renamings. He rewrites some of the σ -calculus laws but cannot pass the confluence checker. Because weakening is a constructor, variables in his representation can be represented by multiple terms that are not definitionally equal, which results in the loss of definitional equalities. Furthermore, he represents substitutions using vectors, and thus would not be able to support the full σ -calculus rule set with proofs.

Wadler and colleagues also investigate the uniform treatment of renamings and substitutions in a simply typed setting [6] which originates from work by McBride [25]. This approach unifies repeated similar lemmas, such as merging all four compositionality lemmas into one. They hide the structural dependence of renamings and substitutions in a uniform, **Mode**-indexed vector (where **V** is the index for renamings and **T** is the index for substitutions) and use termination checking directly to make Agda see the structural dependence when it matters.

Saffrich [27] adopts a different approach and hides the structural dependence behind instance resolution. He works in a scoped syntax setting and represent the uniform structure for renamings and substitutions as functions, similar to the setting in this work. On top he provides reflection-based code generation for the syntax-dependent σ -calculus lemmas, which could possibly be adopted to our setting, again to avoid external code generation.

Adapting our approach with safe rewrite rules to a setting where renamings and substitutions can be treated uniformly would be a logical next step. However, for this to be useful in practice, the equational theory has to be extended in non-obvious ways, to support uniform proofs over renamings and substitutions. Furthermore, in this case we need a two-level rewrite system, one on the type level for normalizing expressions in the lattice of `Modes`, generated by $\mathbb{V} \sqsubseteq \mathbb{T}$ [6], and one for the adapted σ -calculus. When experimenting with this approach, we found that the interaction of the two layers makes preservation of confluence difficult.

6 Conclusion

We have shown how to use the normalization procedure of the σ -calculus to generate definitional equalities in Agda. To this end, we closely followed the underlying equational theory of AUTOSUBST 2, which provides the corresponding normalization procedure for Rocq through propositional equalities. We discovered that, to rewrite the full equational theory and maintain logical consistency, we want two contrary things from substitution: the full computational behavior of a function (so that we can prove the σ -calculus rules from the de Bruijn algebra) and symbolic behavior (so that the rules' left-hand sides remain stuck and qualify for higher-order pattern unification). Opaque blocks reconcile these two demands. We compared our proof of subject reduction for System F to a hand-rolled Agda development and found a substantial reduction in supporting infrastructure (Section 3.6), and discussed the relationship to recent strictification [21] and local-rewrite-rule work [23] (Section 5).

References

- 1 M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '90, page 31–46, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/96709.96712.
- 2 Andreas Abel and Brigitte Pientka. Higher-order dynamic pattern unification for dependent types and records. In Luke Ong, editor, *Typed Lambda Calculi and Applications*, pages 10–26, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- 3 Robin Adams. Formalized metatheory with terms represented by an indexed family of types. In *Proceedings of the 2004 International Conference on Types for Proofs and Programs*, TYPES'04, page 1–16, Berlin, Heidelberg, 2004. Springer-Verlag. doi:10.1007/11617990_1.
- 4 Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type and scope safe universe of syntaxes with binding: Their semantics and proofs, 2021. URL: <https://arxiv.org/abs/2001.11001>, arXiv:2001.11001.
- 5 Guillaume Allais, James Chapman, Conor McBride, and James McKinna. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs*, CPP 2017, page 195–207, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3018610.3018613.
- 6 Thorsten Altenkirch, Nathaniel Burke, and Philip Wadler. Substitution without copy and paste. In *Logical Frameworks and Meta-Languages: Theory and Practice (LFMTP 25)*, Birmingham, UK, 2025.

- 7 Thorsten Altenkirch and Ambrus Kaposi. Type theory in type theory using quotient inductive types. *SIGPLAN Not.*, 51(1):18–29, January 2016. doi:10.1145/2914770.2837638.
- 8 Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. Dedukti: a logical framework based on the $\lambda\pi$ -calculus modulo theory, 2023. URL: <https://arxiv.org/abs/2311.07185>, arXiv:2311.07185.
- 9 Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. Strongly typed term representations in coq. *Journal of Automated Reasoning - JAR*, 49:1–19, 08 2012. doi:10.1007/s10817-011-9219-0.
- 10 Liang-Ting Chen, Fredrik Nordvall Forsberg, and Tzu-Chun Tsai. Can we formalise type theory intrinsically without any compromise? a case study in cubical agda. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP '26*, page 201–215, New York, NY, USA, 2026. Association for Computing Machinery. doi:10.1145/3779031.3779090.
- 11 Jesper Cockx. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In Marc Bezem and Assia Mahboubi, editors, *25th International Conference on Types for Proofs and Programs (TYPES 2019)*, volume 175 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1–2:27, Dagstuhl, Germany, 2020. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2019.2>, doi:10.4230/LIPIcs.TYPES.2019.2.
- 12 Jesper Cockx and Andreas Abel. Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi:10.1145/3236770.
- 13 Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. The taming of the rew: a type theory with computational assumptions. *Proc. ACM Program. Lang.*, 5(POPL), January 2021. doi:10.1145/3434341.
- 14 Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM*, 43(2):362–397, March 1996. doi:10.1145/226643.226675.
- 15 Lawrence Dunn, Val Tannen, and Steve Zdancewic. Tealeaves: Structured monads for generic first-order abstract syntax infrastructure. In *14th International Conference on Interactive Theorem Proving (ITP 2023)*, Leibniz International Proceedings in Informatics (LIPIcs), Dagstuhl, Germany, 2023. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2023.14>.
- 16 Lawrence Dunn, Val Tannen, and Steve Zdancewic. Generic reasoning about first-order syntax with decorated traversable monads. *Journal of Automated Reasoning*, 2025. URL: <https://link.springer.com/article/10.1007/s10817-025-09731-y>.
- 17 Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Phd thesis, Éditeur inconnu, 1972. URL: <https://www.cs.cmu.edu/~kw/scans/girard72thesis.pdf>.
- 18 Daniel Gratzer, Jonathan Sterling, Carlo Angiuli, Thierry Coquand, and Lars Birkedal. Controlling unfolding in type theory. *arXiv preprint arXiv:2210.05420*, 2022. URL: <https://arxiv.org/abs/2210.05420>.
- 19 Martin Hofmann. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory*, pages 13–54. Springer, 1997.
- 20 Jieh Hsiang. Refutational theorem proving using term-rewriting systems. *Artificial Intelligence*, 25(3):255–300, 1985. URL: <https://www.sciencedirect.com/science/article/pii/0004370285900748>, doi:10.1016/0004-3702(85)90074-8.
- 21 Ambrus Kaposi and Loïc Pujet. Type theory in type theory using a strictified syntax. *Proc. ACM Program. Lang.*, 9(ICFP), August 2025. doi:10.1145/3747535.
- 22 Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In Yves Bertot, Temur Kutsia,

- and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1--26:18, Dagstuhl, Germany, 2024. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2024.26>, doi:10.4230/LIPIcs.ITP.2024.26.
- 23 Yann Leray and Théo Winterhalter. Encode the cake and eat it too: Controlling computation in type theory, locally. *Proc. ACM Program. Lang.*, 10(POPL), January 2026. doi:10.1145/3776704.
 - 24 Per Martin-Löf. An intuitionistic theory of types: Predicative part. In H.E. Rose and J.C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 73--118. Elsevier, 1975. URL: <https://www.sciencedirect.com/science/article/pii/S0049237X08719451>, doi:10.1016/S0049-237X(08)71945-1.
 - 25 Conor McBride. Type-preserving renaming and substitution. Unpublished manuscript, 2005. URL: <http://strictlypositive.org/ren-sub.pdf>.
 - 26 John C. Reynolds. Towards a theory of type structure. In B. Robinet, editor, *Programming Symposium*, pages 408--425, Berlin, Heidelberg, 1974. Springer Berlin Heidelberg. URL: <https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/theotypestr.pdf>.
 - 27 Hannes Saffrich. Abstractions for Multi-Sorted Substitutions. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 32:1--32:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl -- Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2024.32>, doi:10.4230/LIPIcs.ITP.2024.32.
 - 28 Hannes Saffrich, Peter Thiemann, and Marius Weidner. Intrinsically typed syntax, a logical relation, and the scourge of the transfer lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development*, TyDe 2024, page 2--15, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3678000.3678201.
 - 29 Steven Schäfer, Gert Smolka, and Tobias Tebbi. Completeness and decidability of de bruijn substitution algebra in coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs*, CPP '15, page 67--73, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2676724.2693163.
 - 30 Steven Schäfer, Tobias Tebbi, and Gert Smolka. Autosubst: Reasoning with de bruijn terms and parallel substitutions. In Christian Urban and Xingyuan Zhang, editors, *Interactive Theorem Proving*, pages 359--374, Cham, 2015. Springer International Publishing. URL: https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_Reasoning.pdf.
 - 31 Jeremy G. Siek and Tianyu Chen. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *Journal of Functional Programming*, 31:e30, 2021. doi:10.1017/S0956796821000241.
 - 32 Kathrin Stark. *Mechanising Syntax with Binders in Coq*. PhD thesis, Saarland University, Saarbrücken, Germany, January 2020. Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.). URL: https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf.
 - 33 Kathrin Stark, Steven Schäfer, and Jonas Kaiser. Autosubst 2: reasoning with multi-sorted de bruijn terms and vector substitutions. CPP 2019, page 166--180, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3293880.3294101.
 - 34 Sandro Stucki. system-f-agda: A formalization of system $f_{<}$ in agda. <https://github.com/sstucki/system-f-agda>, 2017. Accessed 2026.
 - 35 Christian Urban, Stefan Berghofer, and Cezary Kaliszyk. Nominal 2. *Archive of Formal Proofs*, February 2013. <https://isa-afp.org/entries/Nominal2.html>, Formal proof development.

- 36 Christian Urban and Christine Tasson. Nominal techniques in isabelle/hol. In *Proceedings of the 20th International Conference on Automated Deduction, CADE' 20*, page 38–53, Berlin, Heidelberg, 2005. Springer-Verlag. doi:10.1007/11532231_4.
- 37 Philip Wadler. Explicit weakening. *Electronic Proceedings in Theoretical Computer Science*, 413:15–26, November 2024. URL: <http://dx.doi.org/10.4204/EPTCS.413.2>, doi:10.4204/eptcs.413.2.